



**¡Les damos la
bienvenida!**

¿Comenzamos?

Esta clase va a ser

- grabada

a



COMISIÓN N°####

Presentación del equipo

✓ **Profesor/a responsable:** Juan Pérez

✓ **Coordinador/a:** Juan Pérez

✓ **Tutores y tutoras:**

○ Juan Pérez

○ ...

○ ...

○ ...

○ ...

○ ...

○ ...

○ Juan Pérez

○ ...

○ ...

○ ...

○ ...

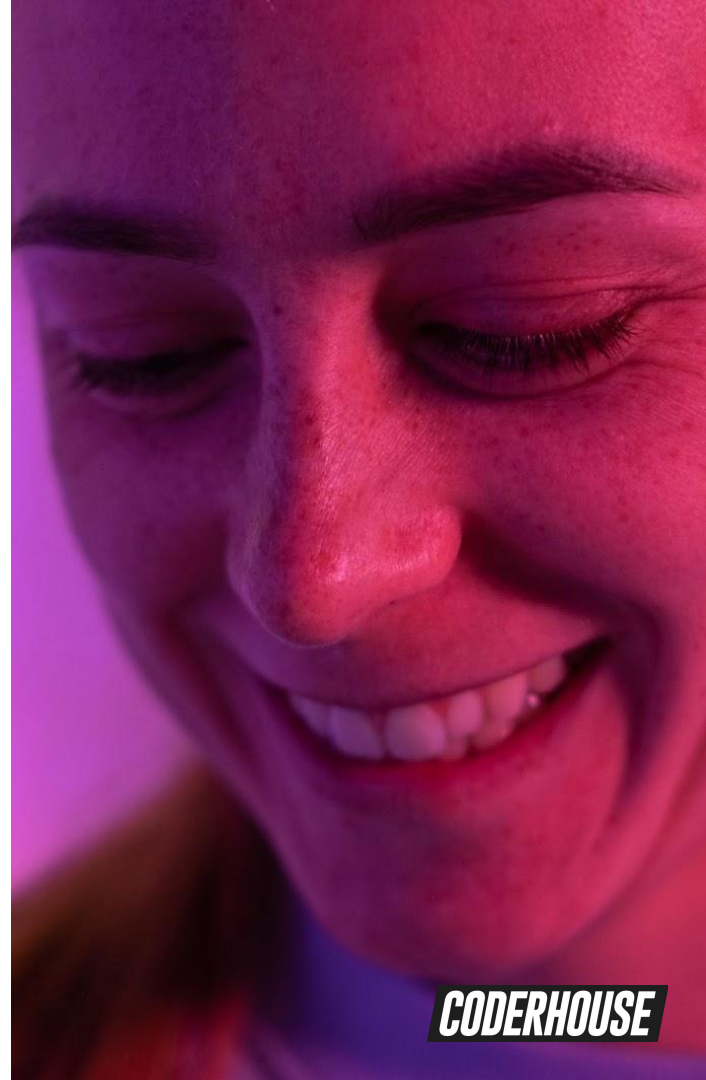
○ ...

○ ...

Presentación de estudiantes

Por encuestas de Zoom

1. País
2. Conocimientos previos
3. ¿Por qué elegiste este curso?



¿Dudas sobre el onboarding?

Míralo aquí



Lo que debes saber
antes de empezar

Acuerdos y compromisos

ACUERDOS Y COMPROMISOS

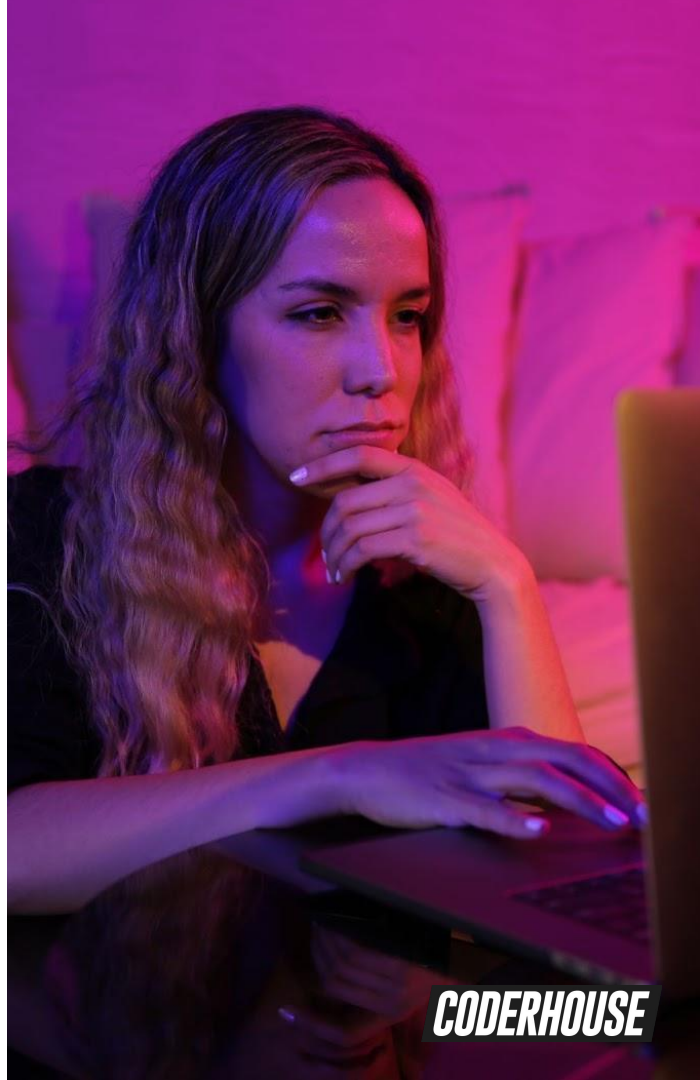
Convivencia

- ✓ Conoce aquí nuestro [código de conducta](#) y ayúdanos a generar un ambiente de clases súper ameno.
- ✓ Durante las clases, emplea los medios de comunicación oficiales para canalizar tus dudas, consultas y/o comentarios: **chat Zoom público y privado, y Slack.**
- ✓ Ten en cuenta **las normas** del buen hablante y del buen oyente, que nunca están de más.
- ✓ Verifica el estado de **la cámara y/o el micrófono** (on/off) de manera que esto no afecte la dinámica de la clase.

ACUERDOS Y COMPROMISOS

Distractores

- ✓ Encuentra tu espacio y crea el momento oportuno para **disfrutar de aprender**
- ✓ Evita dispositivos y aplicaciones que puedan **robar tu atención**
- ✓ Mantén la **mente abierta y flexible**, los prejuicios y paradigmas no están invitados



CODERHOUSE

ACUERDOS Y COMPROMISOS

Herramientas

- ✓ Mantén a tu alcance **agua, mate o café**
- ✓ Si lo necesitas, ten a mano lápiz y papel para que no se escapen las ideas. Pero recuerda que **en Google Drive tienes archivos que te ayudarán a repasar, incluidas las presentaciones.**
- ✓ Conéctate desde algún equipo (laptop, tablet) que te permita **realizar las actividades** sin complicaciones.
- ✓ Todas las clases quedarán grabadas y serán compartidas tanto en la **plataforma de Coderhouse como por Google Drive.**



ACUERDOS Y COMPROMISOS

Equipo

- ✓ **¡Participa de los After Class!** Son un gran espacio para atender dudas y mostrar avances
- ✓ Intercambia ideas por **el chat de Slack**
- ✓ Siempre **interactúa respetuosamente**
- ✓ No te olvides de **valorar tu experiencia** educativa y de contarnos cómo te va

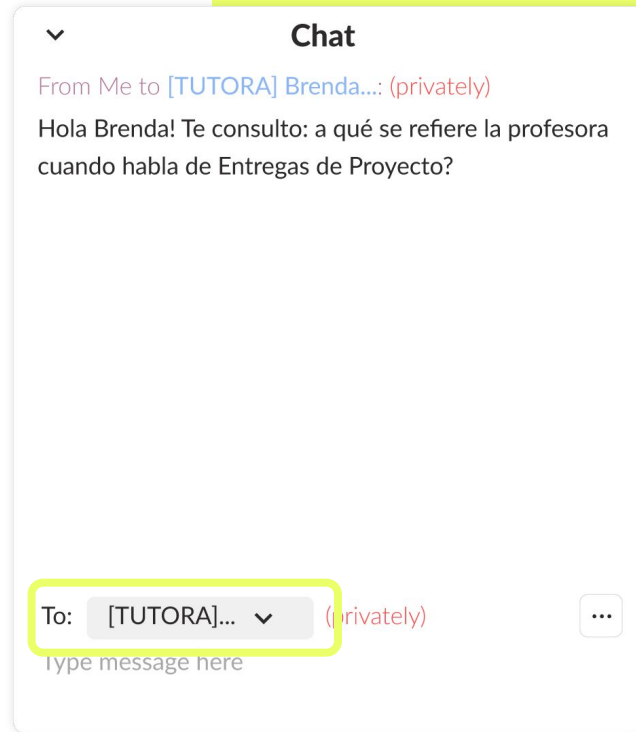
Interacciones en clase

INTERACCIONES EN CLASE

Mientras el profesor explica

Para mantener una comunicación clara y fluida a lo largo de la clase, te proponemos mantener 2 reglas:

1. Si tienes dudas durante la explicación, debes consultarle directamente por privado a tu tutor por el chat de Zoom.

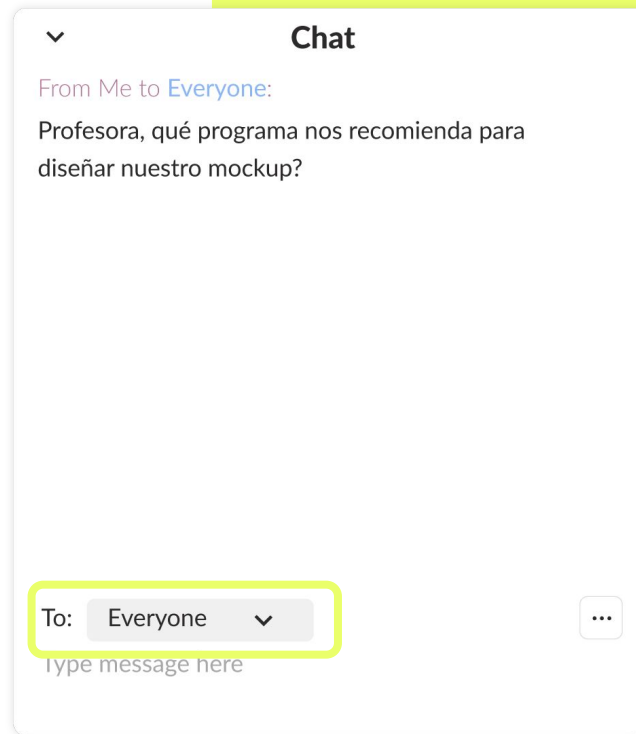


Espacios para consultas

2. Entre contenido y contenido, se abrirán breves espacios de consulta. Allí puedes escribir en el chat tu pregunta.

¡Tu duda puede ayudar a otras personas!

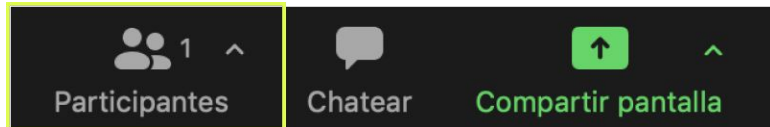
No olvides seleccionar “todos” para que todos puedan leerte (y no solo tu tutor).



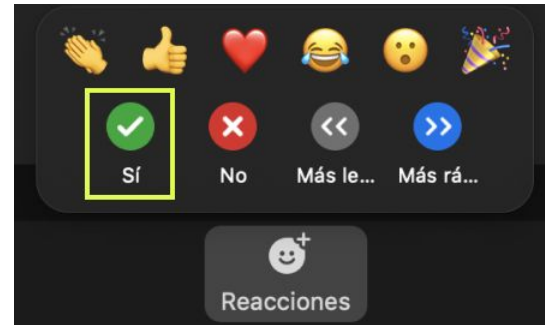
INTERACCIONES EN CLASE

Funcionalidades

Para **evitar saturar el chat de mensajes**, utiliza los signos que figuran en el apartado **Participantes**, dentro de Zoom.**



Por ejemplo: si se pregunta si se escucha correctamente, debes seleccionar la opción "Sí" o "No".



**Para quitar el signo, presiona el mismo botón nuevamente o la opción "clear all".

After Class

AFTER CLASS

¿Qué son?

Te acompañamos para resolver tus consultas sobre el contenido en estos espacios.

Si hay **temas que no se entendieron o necesitan refuerzo** se trabajarán en una clase de 1 hs que opera como **espacio de consulta**.

No son obligatorias ni se toma asistencia, pero son el espacio uno a uno con tu profesor/a** para responder dudas puntuales o reforzar conceptos.

Tu profesor/a está comprometido con tu educación, por lo tanto:

- ✓ Se responderán **dudas puntuales** que hayan quedado sobre los temas dados. ¡Vení preparado, queremos escucharte!
- ✓ Se verán temas de **conocimientos básicos** para la nivelación de saberes.

**Los/as tutores/as también serán protagonistas, liderando 5 veces este espacio en todo el curso.

Desafíos y entregables

DESAFÍOS Y ENTREGABLES

¿Qué son?



Actividades en clase

Ayudan a poner en práctica los conceptos y la teoría vista en clase. No deben ser subidos a la plataforma.



Desafíos entregables

Relacionados completamente con el proyecto final. Deben ser subidos a la plataforma hasta 7 días luego de la clase, para que sean corregidos.

DESAFÍOS Y ENTREGABLES

¿Qué son?



Desafíos complementarios

Desafíos que complementan a los entregables. Son optativos, pero de ser subidos a la plataforma a tiempo, y habiendo sido aprobados, suman puntos para el top 10.

Las prácticas integradoras finalizan con un desafío complementario, no obligatorio.



Entregas del Proyecto final

Entregas con el estado de avance del proyecto final, que deberán ser subidas a la plataforma hasta 7 días luego de la clase para ser corregidas cada tutor/a.

DESAFÍOS Y ENTREGABLES

Proyecto final

El Proyecto final se construye a partir de **los desafíos** que se realizan clase a clase. Se va creando a medida que el estudiante sube los desafíos entregables a nuestra plataforma.

El objetivo es que cada estudiante pueda utilizar su Proyecto final como parte de su portfolio personal.

Se debe subir a la plataforma la ante-última o última clase del curso. En caso de no hacerlo **tendrás 20 días** a partir de la finalización del curso para cargarlo en la plataforma. Pasados esos días el botón de entrega se inhabilitará.



¿Cuál es nuestro
Proyecto final?



PROYECTO FINAL

Backend de una aplicación de e-commerce

Desarrollarás el **backend de una aplicación de e-commerce** para poder vender productos de un rubro a elección.

El servidor se basará en un diseño de capas, orientado a MVC y su código contendrá las estructuras de programación más sólidas por parte del lenguaje ECMAScript.



PROYECTO FINAL

Entrega	Requisito	Fecha
Primera pre entrega	Primera integración de tu PF	Clase 8 – M1
Segunda pre entrega	Profesionalizando la BD de tu servidor	Clase 17 – M1
Tercera pre entrega	Mejorando la arquitectura de tu PF	Clase 30 – M2
Entrega de PF	Proyecto Final	Clase 46 – M3



PROYECTO FINAL

E-commerce con patrones

...

Te dejamos un [modelo de proyecto final](#) (link)

¡Importante!

Los desafíos y entregas se deben cargar **hasta siete días** después de finalizada la clase. Te sugerimos llevarlos al día.



MARTES 25/01 19:00HS - VALORACIÓN REQUERIDA

2. Metodologías de diseño y UX research

DESAFÍO - EXPIRA EL MARTES 01/02/2022 23:59HS

Definiendo el problema, objetivo y solución

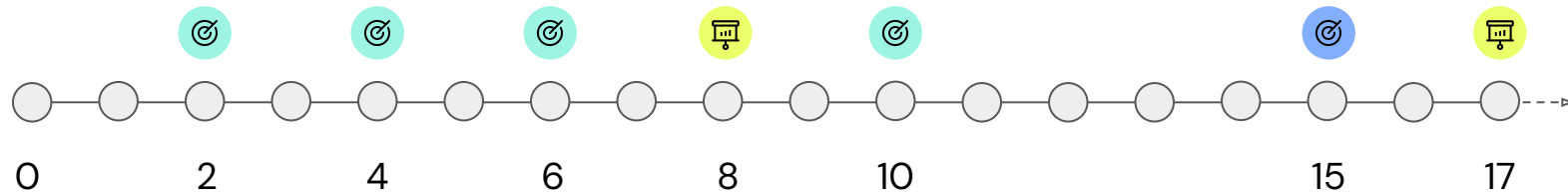
Se bloqueará en 7 días y 11:48hs luego no se podrá entregar.

↑ Entregar

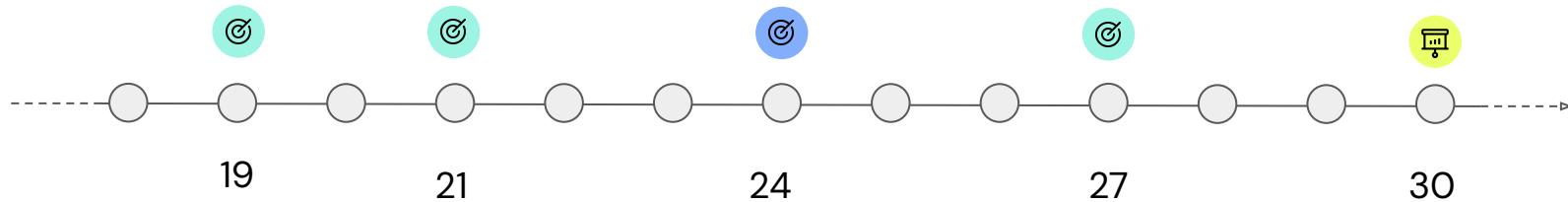


GRILLA DE ENTREGAS

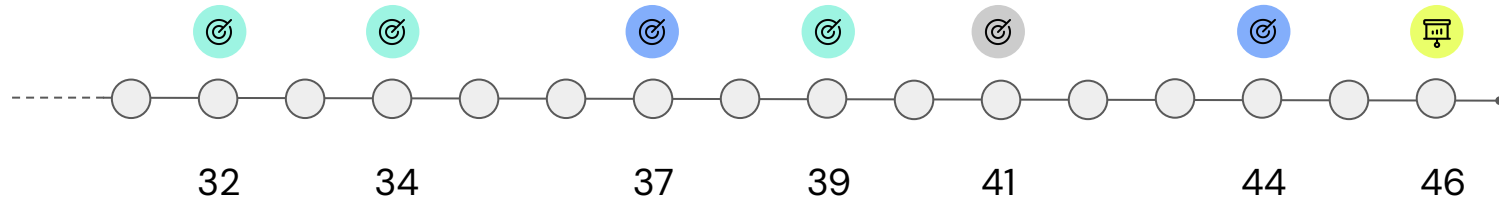
M1



M2



M3



Clase 00. PROGRAMACIÓN BACKEND

Principios de programación Backend

Temario

00

Principios de programación Backend

- ✓ [Programación web](#)
- ✓ [Distintas maneras de probar Javascript](#)
- ✓ [Tipos de datos en JavaScript](#)

01

Principios básicos de JavaScript

- ✓ Tipos de datos y variables en Javascript
- ✓ Javascript y ES6
- ✓ Funciones en Javascript

Objetivos de la clase



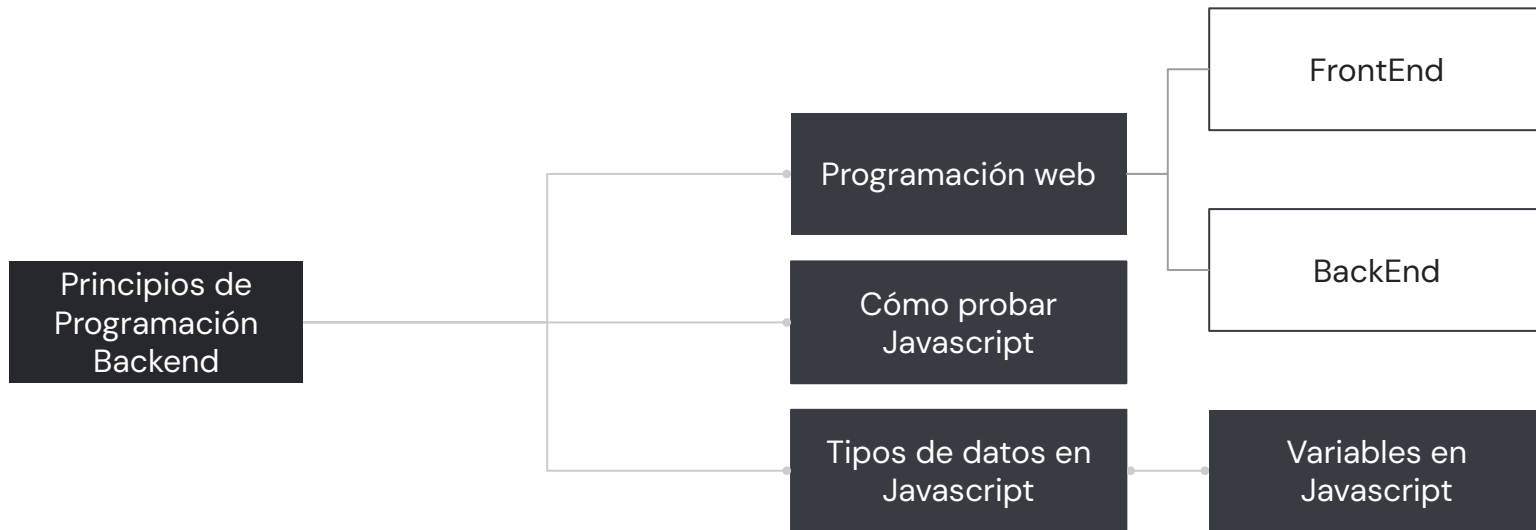
Conocer las diferencias entre programación Front end y Back end



Familiarizarse con las nociones básicas para programar utilizando Javascript y el MERN Stack



MAPA DE CONCEPTOS

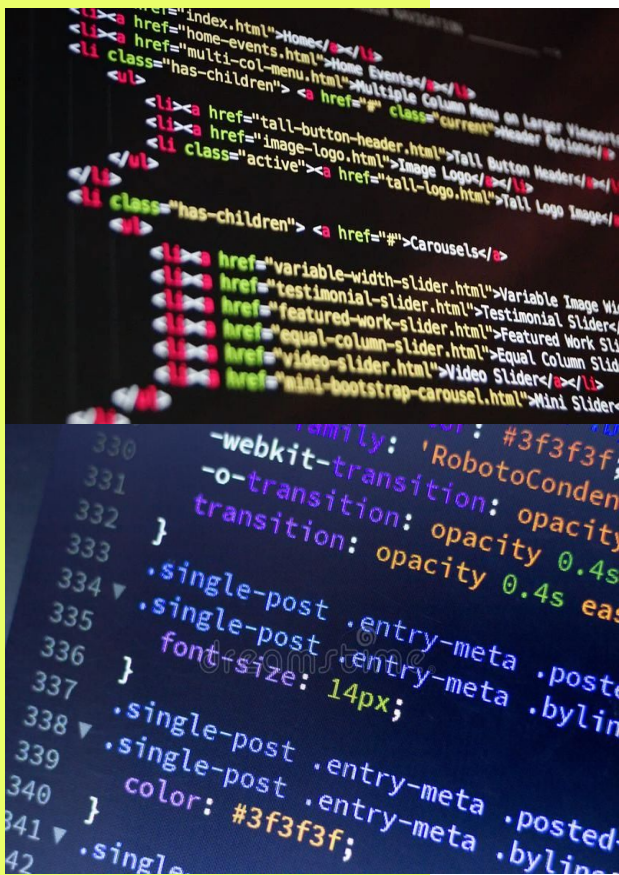


Programación web

¿Qué es el desarrollo web?

Cuando hablamos de desarrollo web, nos referimos a la construcción y mantenimiento de estos sitios que pueden encontrarse en la World Wide Web (www) y sus redes derivadas.

Originalmente éste sólo se basaba en el desarrollo de páginas estáticas. Sin embargo, con el paso del tiempo estas fueron requiriendo más dinamismo, al punto de comenzar a desarrollar no sólo páginas estáticas, sino **sitios web completos** o incluso **aplicaciones web**.



Las dos caras de una misma moneda

Conforme las necesidades del desarrollo web van creciendo, surge la necesidad de comenzar a separar el desarrollo en dos aspectos importantes: **FrontEnd** y **BackEnd**.

FrontEnd y BackEnd

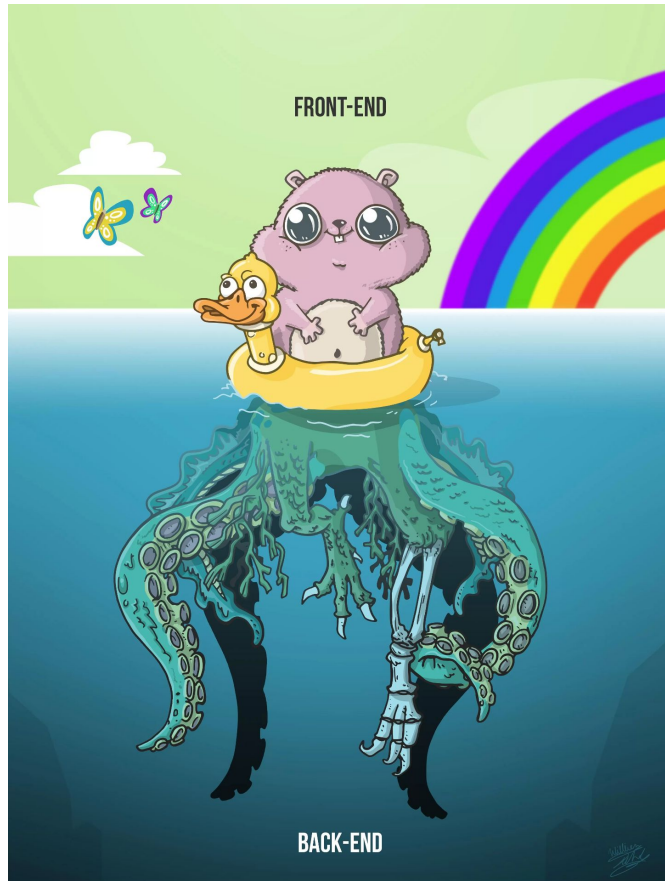
FrontEnd y Backend

El **FrontEnd** comprende toda la parte **visual y de interacción directa** con el usuario. Su fin es dar la mejor impresión al usuario sobre la página o aplicación.

- ✓ Imágenes
- ✓ Colores
- ✓ Botones
- ✓ Interacciones

El **BackEnd** comprende toda la parte **lógica y de manejo de información**. Es decir, la base lógica de nuestra página o aplicación. El usuario NO sabe lo que hay detrás.

- ✓ El almacenamiento de información
- ✓ Los cálculos complejos
- ✓ Los servidores donde viven las páginas
- ✓ Manejo de información en general



Mientras que el **FrontEnd** se centra en la experiencia del usuario, ¡que esta sea lo más presentable y amigable posible!

...el **BackEnd** tiene una estructura interna para que todo funcione correctamente. ¡Que no lo vea el usuario!

**Ahora... ¿frontend vs.
backend?**

¿FrontEnd vs BackEnd?

A menudo escuchamos a otras personas decir “odio el BackEnd” o “qué aburrido el frontEnd”. Esto ha polarizado a la comunidad del desarrollo, pensando que el mundo sería mejor si alguno de los dos no existiera.

¡FALSO! Ambos necesitan el uno del otro para poder estructurar páginas y aplicaciones. De esta manera, podremos resolver los problemas del mundo real con soluciones tecnológicas.

¡Hay que hacer las paces!

Stack MERN

¿Qué es un “stack”?

Son ciertas tecnologías que, en conjunto, nos brindarán la posibilidad de desarrollar sistemas completos, debido a su máxima compatibilidad.

Podríamos decir que es la forma en la que el FrontEnd y el backEnd hacen las pases, ya que trabajan en conjunto. En este curso trabajaremos sobre el Stack MERN.

Componentes de MERN Stack



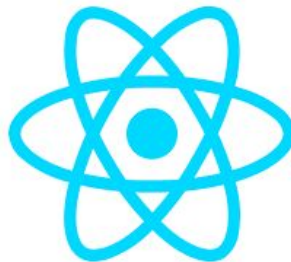
MongoDB

base de datos no
relacional



ExpressJS

framework para crear
servidores en NodeJS



ReactJS

librería para desarrollar
interfaces de usuario



el corazón de todo

NodeJS

entorno de ejecución de
Javascript

Distintas maneras de probar Javascript

¿Cómo probamos Javascript?

Cliente web

Lo utilizamos desde la consola del navegador, no necesitamos instalar nada ya que todo el motor vive en el navegador

Node JS

Debemos instalarlo ya que se usa fuera del navegador, por lo que podremos escribir Javascript desde nuestro computador directamente. Éste es el que utilizaremos a lo largo del curso.

Cliente web

Se utiliza desde cualquier navegador. Algunos de los comandos que más usarás y verás a lo largo de un debug en cliente web son:

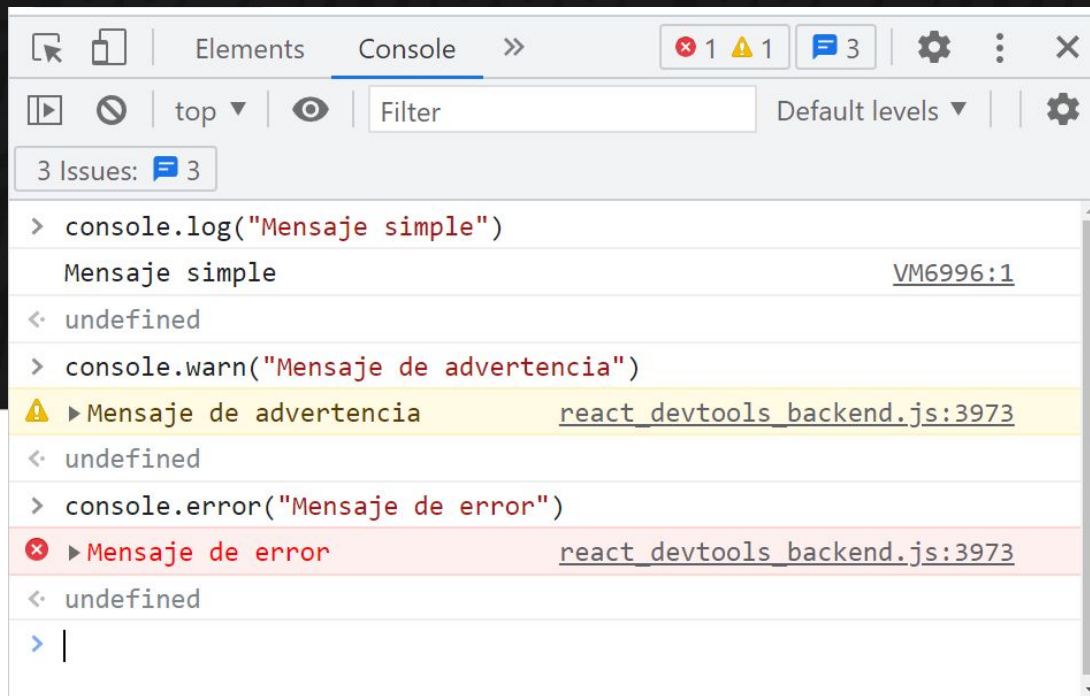
- ✓ **`console.log("texto")`** Mostrará un texto simple
- ✓ **`console.warn("texto")`** Mostrará una advertencia
- ✓ **`console.error("texto")`** Mostrará un error
- ✓ **`console.clear()`** Limpiará la consola para evitar que se acumule el código.



Ejemplo en vivo

Consola web: object (.log, .error, .warn y .clear)

Consola de cliente web



Node js

Cuando trabajamos en backend, no tenemos un navegador, es por ello que se precisa de alguna tecnología que nos permita correr código de Javascript, sin necesidad de abrirlo en el navegador. Ahí es cuando utilizamos Node js, para ello, toca considerar:

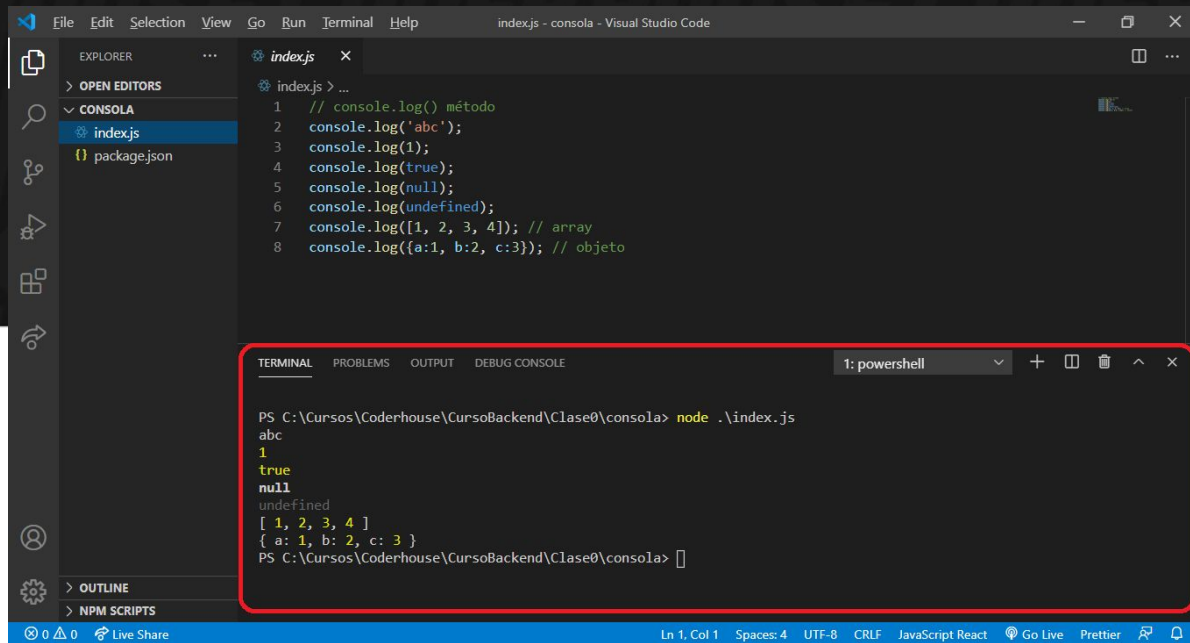
- ✓ Node js levantará un entorno completo para probar nuestras funciones.
- ✓ Debe correrse desde un CLI para poder visualizar los avances del código.
- ✓ Usualmente usaremos el CLI de Visual Studio Code.



Ejemplo en vivo

Consola CLI en Visual Studio Code

Código de Javascript corriendo desde Node js



The screenshot shows the Visual Studio Code interface with a file named `index.js` open in the editor. The file contains the following JavaScript code:

```
1 // console.log() método
2 console.log('abc');
3 console.log(1);
4 console.log(true);
5 console.log(null);
6 console.log(undefined);
7 console.log([1, 2, 3, 4]); // array
8 console.log({a:1, b:2, c:3}); // objeto
```

The terminal window at the bottom shows the command `node .\index.js` being executed in a PowerShell prompt. The output of the command is:

```
PS C:\Cursos\Coderhouse\CursoBackend\Clase0\consola> node .\index.js
abc
1
true
null
undefined
[ 1, 2, 3, 4 ]
{ a: 1, b: 2, c: 3 }
PS C:\Cursos\Coderhouse\CursoBackend\Clase0\consola>
```



Para pensar

¿Cuándo utilizarías un cliente web, y cuándo utilizarías Node js al utilizar Javascript?

¿Una es reemplazo de la otra, o podríamos considerarlas complementarias?



Break

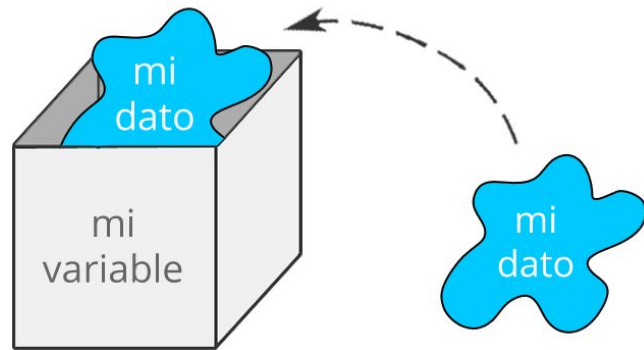
¡10 minutos y volvemos!

Tipos de datos en Javascript

Tipo de dato

Es el atributo que especifica la clase de dato que almacena la variable.

Especifica con qué estaremos trabajando, para que la computadora reconozca qué operaciones puede hacer con él.



Tipos de datos

✓ **Tipo Primitivos:** Incluyen a las cadenas de texto (String), variables booleanas cuyo valor puede ser true o false (Boolean) y números (Number). Además hay dos tipos primitivos especiales que son Null y Undefined. La copia es por valor.

✓ **Tipo Objeto:** Incluyen a los objetos (Object), a los arrays (Array) y funciones. La copia es por referencia.

		Nombre	Descripción
Tipos de datos en Javascript	Tipos primitivos	String	Cadenas de texto
		Number	Valores numéricos
		Booleans	True/false
		Null	Tipo especial, contiene null
		Undefined	Tipo especial, contiene undefined.
	Tipos objeto	Tipos predefinidos de Javascript	Date (fecha) RegExp (expresiones regulares) Error (datos de error)
		Tipos definidos por el programador/usuario	Funciones simples Clases
		Arrays	Serie de elementos o formación tipo vector o matriz. Lo consideraremos un objeto especial que carece de métodos.
		Objetos especiales	Objeto global
			Objeto prototipo
			Otros

Variables en Javascript

Variables en Javascript

Ahora que hemos comprendido cuáles son los tipos de datos, es importante comprender dónde serán guardados dichos datos. Una variable es un **espacio de memoria** apartado por la computadora, para poder **guardar un dato**.

Tal cual lo dice su nombre, una variable puede **cambiar su valor** si así el programa lo necesita. Ello permite la reutilización de una sola variable, para los casos que se vayan requiriendo.



Datos y variables

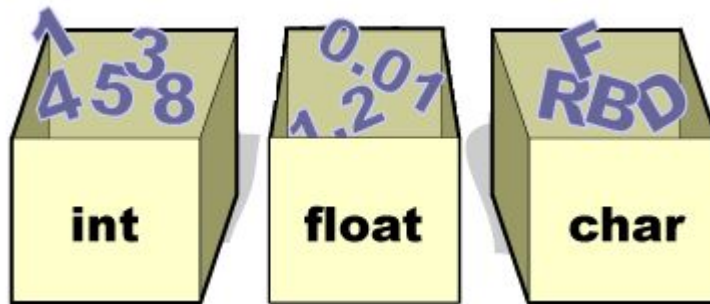
Duración: 10 min



ACTIVIDAD EN CLASE

Datos y variables

Definir variables que almacenen datos (nombre, edad, precio, nombres de series y películas), mostrar esos valores por consola, incrementar la edad en 1, una serie a la lista y volver a mostrarlas. Compartir la definición en el Visual Studio Code.



¡Atención!

Recuerda instalar el IDE Visual Studio Code
para la próxima clase.



¿Preguntas?

Opina y valora
esta clase

Resumen de la clase hoy

- ✓ Programación web
- ✓ Distintas maneras de probar Javascript
- ✓ Tipos de datos en Javascript
- ✓ Variables en Javascript

Muchas gracias.

#DemocratizandoLaEducación